

Task 2.4 — Counters: JMH + Scalability

Данил

1 Экспериментальная установка

Измерения производительности выполнены с помощью JMH в режиме `Throughput` (единицы `ops/ms`). Использовались сценарии:

- `get-only`: benchmark вызывает только `get()`
- `increment-only`: benchmark вызывает только `increment()`

Сравнивались реализации счётчика:

- `ThreadUnsafeCounter` (без синхронизации)
- `LockedCounter` на `ReentrantLock` (`fair=false` и `fair=true`)
- `SplitCounter` (lock splitting) с разной гранулярностью

Про «не оптимизировал ли компилятор нагрузку» (DCE). JMH старается защищать измерения от выбрасывания «мертвого» кода. Дополнительно в тестовом наборе (и при построении графиков) используется реальное чтение результата `get()`, что делает эффект вычислений наблюдаемым, а значит их нельзя просто выкинуть как ненужные.

2 Easy

2.1 ThreadUnsafeCounter

Реализация:

- `increment(): data++`
- `get(): return data`

Корректность. `ThreadUnsafeCounter` не потокобезопасен: операция `data++` не атомарна (чтение → увеличение → запись). Поэтому при конкурентных инкрементах часть обновлений может теряться, а `get()` может наблюдать состояния, которые не соответствуют «идеальному» последовательному счётчику.

Масштабируемость. В `get-only` обычно масштабируется хорошо, т.к. это просто чтение переменной. В `increment-only throughput` может выглядеть высоким, но корректность результата не гарантируется.

2.2 NoContentionBaseline

NoContentionBaseline запускает N потоков, где каждый поток увеличивает **свой** приватный счётчик. Это baseline без contention: общих данных нет, поэтому при увеличении числа потоков throughput растёт, пока не исчерпаны аппаратные ресурсы CPU.

2.3 LockedCounter: ReentrantLock

LockedCounter использует один общий ReentrantLock для защиты переменной data. Рассмотрены два режима:

- **unfair** (fair=false)
- **fair** (fair=true)

Масштабируемость. Обе реализации масштабируются плохо: все потоки конкурируют за один lock, поэтому появляется узкое место и throughput быстро выходит на плато.

Oversubscription. При числе потоков больше числа CPU рост throughput прекращается и часто появляется деградация: увеличиваются накладные расходы планировщика и число переключений контекста.

Fair vs Unfair. fair=true обычно медленнее: требуется поддерживать справедливую очередь ожидания, что увеличивает парковки/пробуждения потоков и scheduler overhead. fair=false чаще позволяет «перехватывать» lock потоку, который уже на CPU, поэтому может давать больший throughput.

2.4 Выводы Easy

- Между unsafe и lock-based реализациями есть разрыв по single-thread throughput из-за стоимости lock/unlock.
- get масштабируется лучше, чем increment (особенно в lock-based вариантах).
- Один общий lock (LockedCounter) почти сразу становится bottleneck.
- fair=true обычно снижает throughput из-за дополнительных издержек планировщика.

3 Medium

3.1 SplitCounter (lock splitting)

SplitCounter использует массив «порций» portions[] и массив lock'ов locks[]. Индекс порции выбирается по идентификатору потока:

$$idx = threadId \bmod granularity$$

increment. increment() блокирует ровно один lock (locks[idx]) и увеличивает portions[idx].

get. `get()` блокирует **все** `locks`, суммирует все порции и затем освобождает `locks`. Это даёт согласованный снимок: на время `get` инкременты блокируются.

3.2 Корректность и тесты

Почему корректно.

- Каждый инкремент защищён `lock`'ом соответствующей порции, поэтому обновления не теряются внутри порции.
- `get` удерживает все `locks` на время суммирования, поэтому результат соответствует согласованному состоянию «между» операциями `increment`.

Тесты. Добавлены тесты корректности, включая стресс-тесты:

- `single-thread`: точная сумма после многих инкрементов;
- стабильность `get` без инкрементов;
- стресс: много потоков, точная сумма после завершения;
- стресс: конкурентные `get` во время инкрементов;
- стресс: `oversubscription` + таймаут против зависаний.

3.3 Масштабируемость и `oversubscription`

`increment` масштабируется лучше. За счёт распределения по порциям уменьшается `contention`: потоки не стоят в одной очереди к одному `lock`.

`get` масштабируется хуже. `get` создаёт глобальную точку синхронизации (`lock-all`), поэтому параллелизм не помогает. Кроме того, при большой гранулярности стоимость `get` растёт из-за необходимости захвата множества `lock`'ов.

`Oversubscription`. При потоков больше `CPU throughput` перестает расти (или падает), так как увеличиваются переключения контекста. Для `increment splitting` обычно переносит `oversubscription` лучше, чем один `lock`, но рост всё равно ограничен `CPU`.

3.4 Коллизии порций и аналогия

Можно искусственно увеличить `contention`, если выбрать маленькую гранулярность относительно числа потоков: многие потоки будут попадать в одну порцию, и выигрыш от `splitting` снизится.

Аналогия в однопоточных структурах: **`hash table`** с коллизиями (несколько ключей в одном `bucket`). Как уменьшить проблему:

- увеличить `granularity` (больше «`buckets`»);
- улучшить распределение потоков по порциям (уменьшить перекоп);
- использовать динамическое расширение (как делают реальные структуры).

4 Графики производительности

Ниже приведены графики throughput (ops/ms) в зависимости от числа потоков. Error bars взяты из `scoreError` JMH.

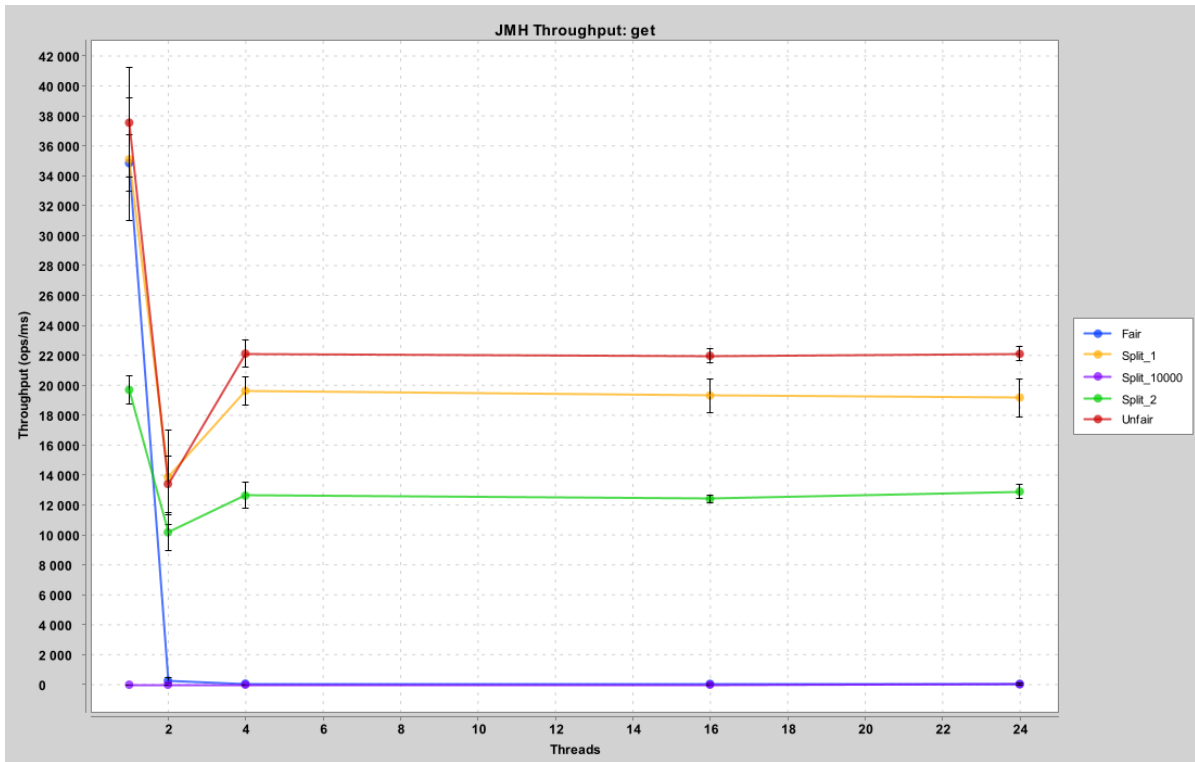


Рис. 1: Throughput vs Threads: get-only

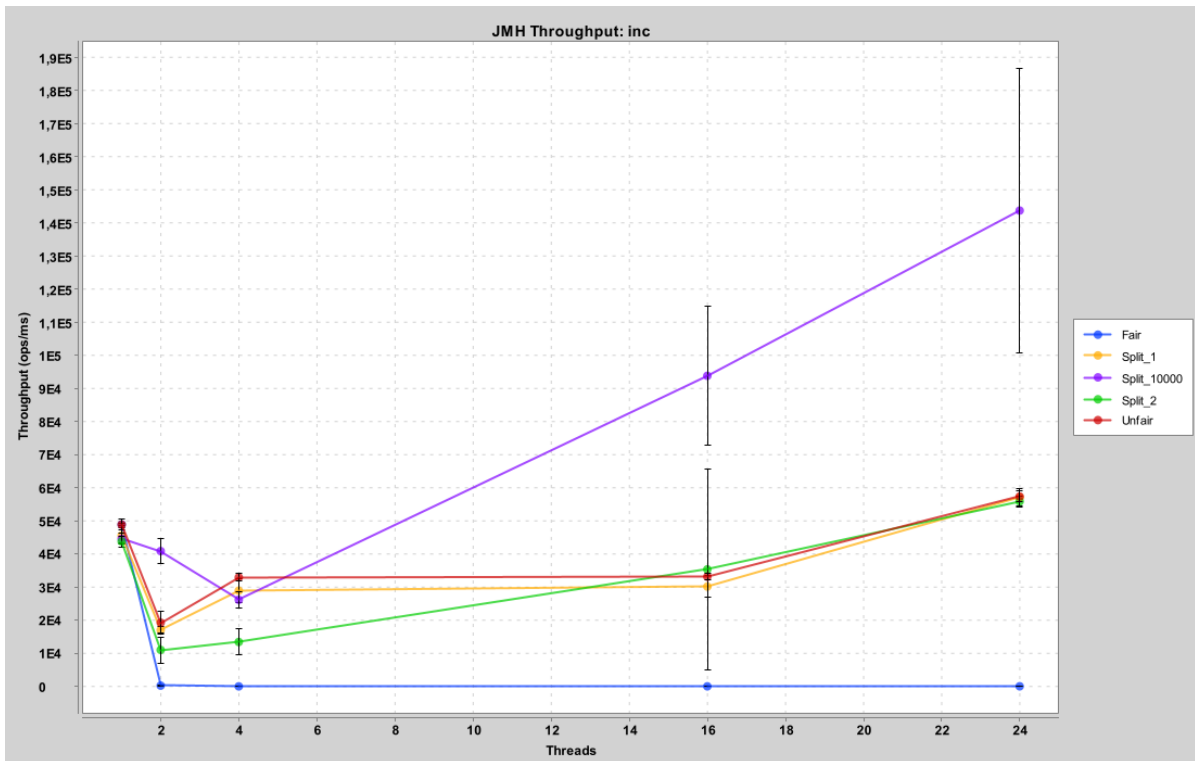


Рис. 2: Throughput vs Threads: increment-only

5 Итоговые выводы

- `ThreadUnsafeCounter` даёт высокий `throughput`, но не гарантирует корректность при конкурентных инкрементах.
- `LockedCounter` корректен, но плохо масштабируется из-за `contention` на одном `lock`.
- `fair=true` обычно снижает `throughput` за счёт `overhead` планировщика и дополнительных переключений контекста.
- `SplitCounter` улучшает масштабируемость `increment`, но делает `get` дороже (`lock-all`).